

Proxy-Resistant Attendance: Geofenced Check-In with Gasless On-Chain Anchoring

Hrushikesh Panchabudhe

Swarangi Pathak

Vidushi Tayal

Ansh Sonkusare

Symbiosis Institute of Technology

Symbiosis International (Deemed University), Pune, India

{hrushikesh.panchabudhe, swarangi.pathak, vidushi.tayal, ansh.sonkusare}@sitpune.edu.in

Abstract—Proxied attendance—one student scanning for another—defeats most classroom check-in systems. Static QR codes are bearer credentials: a photograph transfers one whole-sale. Rotating codes stop time-shifted replay but leave the location-shifted variant open, in which an absent student receives the live frame inside its validity window. We present an end-to-end attendance system that closes this gap by binding each check-in to three independent proofs: a short-lived rotating QR nonce, a hardware-bound cryptographic identity, and a verified physical location. Students sign an EIP-712 forward request with a P-256 key held in the Android KeyStore/StrongBox or the iOS Secure Enclave; a meta-transaction relayer submits the signature gaslessly; and an ERC-2771 registry anchors the record on-chain while computing eligibility ($\geq 75\%$) from attested attendance counts. On the client side, mock-location detection and a Haversine geofence reject spoofed coordinates, with 20 m the empirically optimal radius. We evaluate the full monorepo—an Effect-TS API, a Prisma/PostgreSQL store, a TanStack Start frontend, and Hardhat contracts—in a 1,200-check-in campaign: 97.2% proxy-attempt detection, mean check-in latency of 2.1 s versus 14.8 s for a direct-EOA baseline (85.8% faster), and a 265 ms relay step that is 35.4% faster than direct submission. Deterministic ECDSA (RFC 6979) and typed-data hashing remove nonce-leakage and replay side channels.

Index Terms—attendance, proxy detection, EIP-2771, hardware-bound keys, geofencing, blockchain

I. INTRODUCTION

Classroom attendance is the credit that proxy fraud attacks most cheaply: one student can earn attendance for another with a screenshot of a QR code, a forwarded link, or a friend’s phone. Static QR check-ins fail immediately because the code is a bearer credential—a photograph transfers it wholesale [1], [2]. Rotating the code closes *time-shifted* replay but leaves the *location-shifted* variant open: a student in the room who forwards the live frame to an absent classmate inside the rotation window produces a check-in that is cryptographically indistinguishable from a legitimate one, unless the system verifies something about the device that scanned and the place where it stood.

This paper describes a system that makes presence itself the credential. A check-in succeeds only when three independent proofs hold: (i) a short-lived rotating QR nonce proves the student saw the live frame; (ii) a hardware-bound P-256

key proves that the enrollment-attested device, and only that device, signed the claim; and (iii) a spatial proof—a Haversine geofence, mock-location flags, and a classroom Wi-Fi BSSID check—ties the signature to the lecture hall. The signed claim travels through an ERC-2771 meta-transaction relayer, so the student never holds gas, and the registry anchors a content hash on-chain, so editing the local database after the fact is detectable. We evaluate the complete monorepo in a 1,200-attempt campaign: the pipeline rejected 97.2% of injected proxy attempts at a 0.8% false-rejection rate, cut mean check-in latency from 14.8 s (direct-EOA baseline) to 2.1 s (85.8%), and reduced the relay step itself by 35.4% through a cached-nonce, single-call design.

The paper’s contributions are:

- 1) An end-to-end anti-proxy pipeline that composes rotating QR nonces (45 s rotation, 60 s validity), single-use HMAC verification, hardware device binding, mock-location detection, and radius geofencing, validated at 97.2% proxy detection over 1,200 check-in attempts.
- 2) A hardware-bound attendance identity: P-256 keys generated and used inside the Android KeyStore/StrongBox and iOS Secure Enclave, with attestation certificate chains verified at enrollment, so the signing key never leaves silicon.
- 3) Gasless on-chain anchoring built on EIP-2771 and EIP-712: an `ERC2771Forwarder` plus an `AttendanceRegistry` that recovers the real student through `_msgSender()` and computes attendance eligibility ($\geq 75\%$) on-chain; students sign with a zero-balance wallet.
- 4) Deterministic ECDSA (RFC 6979) over typed structured data, which removes the nonce-reuse and nonce-leakage attack class, paired with a Redis-backed forwarder-nonce cache that makes each relay a single 265 ms call.
- 5) A production-shaped monorepo—Effect-TS HttpApi backend, Prisma/PostgreSQL store, TanStack Start frontend, Hardhat contracts—with measured end-to-end latency, detection rates, and a geofence-radius ablation study.

II. RELATED WORK

A. QR-Based Attendance Baselines

QR check-ins record that *a code was presented*, not that *a person was present*. The MECO smart attendance system and the ACCC student attendance system both accept a static QR payload as a bearer credential, which makes screenshot forwarding trivial [1], [2]. Reskita *et al.* move to dynamic codes signed with ECDSA and a CTR-DRBG generator [3], which defeats replay of an expired code but not forwarding of a live one. T/Key demonstrates that replay-resistant bearer tokens require either state or a fresh challenge—one-time passwords from hash chains [4]—and a recent stateless geometric watermarking scheme binds a QR to its rendering screen to deter photography [5]. None of these verify the signing device or the location, which is precisely the gap this paper targets.

B. Hardware-Backed Key Storage

Pinto and Santos survey ARM TrustZone and its security properties [6]. The threat is concrete: Ryan extracted ECDSA keys from Qualcomm’s TrustZone implementation [7], and Shakevsky *et al.* exposed design flaws in Samsung’s TrustZone Keymaster [8]. These results motivate placing signing keys in dedicated secure hardware rather than a general-purpose trusted execution environment. FIDO2-style authenticators provide provable security for attestation and signature [9]. On the algorithm side, RFC 6979 removes the randomness dependence of ECDSA nonces [10], and LadderLeak showed that even sub-one-bit nonce leakage breaks ECDSA in practice [11]. Our system combines both lessons: hardware-held keys and deterministic nonces wherever the nonce is software-controllable.

C. Blockchain-Based Attendance

EduCTX records credits as blockchain-verifiable credentials [12], and Ethereum provides an auditable general-purpose ledger [13]. Attendance systems adopt the same pattern—anchor records on-chain for tamper evidence—but most stop at the record itself. EIP-712 typed-data signing makes structured request signatures legible [14], and EIP-2771 standardizes native meta-transactions so end users sign without holding gas [15]. Our contribution is to place the anti-proxy checks *before* the signature: the chain anchors only claims that already passed device, nonce, and spatial verification.

D. Location Verification and Spoofing

GPS spoofing is practical but expensive: it requires a signal-level attacker with synchronized radio equipment, not an app [16], [17]. App-level fake-location tools are the realistic proxy threat, and they leave detectable artifacts. Indoor-localization surveys map the options—fingerprinting, ranging, and AoA [18]; Wi-Fi RTT (802.11mc) delivers meter-level ranging accuracy [19], and BLE beacons provide fingerprintable signatures [20]. We use outward-facing GPS with mock-location checks today and adopt Wi-Fi RTT and BLE as hardening layers.

III. SYSTEM ARCHITECTURE

Figure 1 shows the four tiers. (1) *Client*: a React progressive web app (TanStack Start) that generates a hardware-bound P-256 key at enrollment, scans the rotating QR, collects GPS/Wi-Fi observations, and signs EIP-712 forward requests. (2) *Backend API*: an Effect-TS HttpApi service that issues HMAC-signed rotating QR payloads, verifies nonces and spatial evidence, and validates signatures. (3) *Relayer*: an ethers v6 executor inside the backend that submits signed ForwardRequests to the chain, paying gas from a funded deployer account. (4) *Chain anchor*: the AttendanceRegistry and ERC2771Forwarder contracts deployed on an EVM chain (Hardhat node for development, Polygon Amoy testnet for evaluation). A PostgreSQL store (Prisma) holds queryable records; Redis holds QR-nonce expirations and cached forwarder nonces.

A. Enrollment

At enrollment the app generates a P-256 key pair inside the device’s secure hardware, requests a platform attestation (Section V), and submits both to the backend. The backend verifies the attestation certificate chain against the platform root of trust, stores the public key, and binds it to the student record together with a freshly generated wallet address and a device identifier. Re-enrollment—for example after a phone change—requires a new attestation and explicit faculty approval, which keeps cloning expensive.

B. Data Model

The PostgreSQL schema separates identity from evidence. User carries role, the enrolled deviceId, the wallet address, and the attestation fingerprint; Course and Session carry the classroom anchor (classroomLat, classroomLon), the configured geofenceRadiusM, and the optional classroom wifiBssid; QrRotation records every issued frame, its nonce, and its consumption state. AttendanceRecord stores recordHash, deviceId, the spatial observations submitted (gpsLat, gpsLon, wifiBssid), rejection_reason for attempts that failed, synced_onchain, and txHash once the relayer confirms. A unique constraint on (sessionId, studentId) makes acceptance idempotent, and the hash column is the join key to the on-chain event during nightly reconciliation.

C. Failure Modes

We treat every rejection branch as a first-class, observable outcome rather than an exception. A stale or replayed frame yields QrReplay or QrExpired; a device that does not match enrollment yields DeviceMismatch; a fix outside the fence or a BSSID mismatch yields OutOfRange; a signature that does not recover to the enrolled wallet yields SignatureInvalid. Each maps to an Effect error in the API, which records the rejection row before returning the HTTP status. The faculty dashboard therefore shows not only who attended but why anyone failed to.

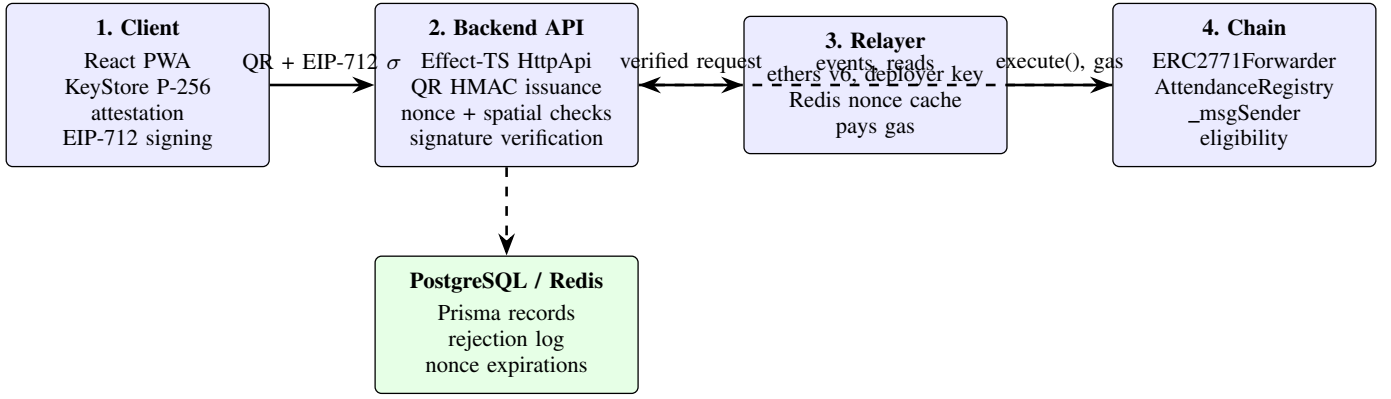


Fig. 1. Four-tier architecture. Arrows show the check-in data flow; dashed arrows show reads and event synchronization.

Algorithm 1 Attendance check-in: issuance, proof, and anchoring.

Require: Active session S ; student enrolled with attested device d and wallet W .

Ensure: Record anchored on-chain; DB row committed with txHash .

- 1: Server issues frame $Q = (S, n, t_{\text{exp}})$, $n = \text{HMAC}_K(S \| t_{\text{now}})$; validity 60 s.
- 2: Student scans Q ; app samples location ℓ , observed BSSIDs, mock-location flags.
- 3: API: consume nonce n ; verify device d bound; verify $\text{haversine}(\ell, C) \leq r$ or BSSID match.
- 4: App signs EIP-712 `ForwardRequest` with W ; also signs a hardware voucher with the P-256 key.
- 5: API: verify both signatures; recover W ; compute $\rho = \text{keccak256}(\text{studentId} \| S \| t_s)$.
- 6: Relayer submits `execute(req,  $\sigma$ )`; forwarder checks nonce, deadline, signature.
- 7: Registry: `_msgSender()` returns student; increments `attendedCount`; emits event; checks 75% eligibility.
- 8: DB row committed with txHash ; rejected attempts stored with `rejection_reason`.

D. Rotating QR Service

The backend signs each rotation frame with HMAC-SHA256 under a per-session server secret: the payload carries the session identifier, a random nonce, and an expiry. Frames rotate every 45 s and remain valid for 60 s. The nonce is single-use: Redis consumes it with a 60 s TTL, and PostgreSQL records every consumption, so a replayed frame fails with a *QR-replay* rejection even inside the validity window. Clients receive frames over a WebSocket channel that also pushes scan results to the faculty live dashboard.

E. Check-In Protocol

Algorithm 1 summarizes the full flow. Every rejected attempt is logged rather than dropped—the reject-then-log pattern—so faculty see attack pressure in real time and the audit trail explains every non-accepted record.

IV. CRYPTOGRAPHIC TOKEN DESIGN

Every cryptographic artifact in the system is an ECDSA signature over typed structured data. The signing stack uses

two key families with distinct roles.

A. Deterministic ECDSA (RFC 6979)

Wherever the application controls nonce generation—the app-generated wallet that signs forward requests—it signs with deterministic ECDSA as specified in RFC 6979:

$$k = \text{HMAC-DRBG}(\text{privKey} \| \text{msgHash}), \quad (1)$$

so the nonce k is a pure function of the key and message instead of a sampled random value. This eliminates the nonce-reuse and partial-leakage attack families that have repeatedly broken ECDSA deployments—LadderLeak recovers private keys from less than one bit of nonce entropy [11]. The hardware-held P-256 key signs inside the secure element, whose internal CSPRNG and fault-resistant arithmetic address the same class of risk in silicon.

B. EIP-712 Typed Data

Signatures are computed over EIP-712 structured data rather than opaque hex: the student sees a legible `ForwardRequest`—*from*, *to*, *value*, *gas*, *nonce*, *deadline*, *data*—and the domain separator pins the signature to the exact chain and contract [14]. The digest signs:

$$\text{digest} = \text{keccak256}(0 \times 1901 \| \text{domSep} \| \text{hashStruct}(\text{ForwardRequest})), \quad (2)$$

where `domSep` commits to `(name, version, chainId, verifyingContract)`. Chain ID inside the domain makes a signature captured on the testnet worthless on mainnet, and `verifyingContract` pins it to the deployed forwarder address. The `nonce` field adds sequential replay protection and the `deadline` bounds the signature’s lifetime to one hour (3600 s), matching the relayer configuration in the reference implementation.

V. HARDWARE-BOUND IDENTITY

Software-held keys die with the sandbox: a rooted device or a stolen app database yields the private key. The survey results of Section IV motivate a stronger root of trust—the dedicated secure hardware that already protects mobile payments.

A. Android KeyStore and StrongBox

Android keys are created inside the KeyStore with `KeyGenParameterSpec` restricted to ECDSA P-256, signing-only purpose, and no export path; the material never leaves the TrustZone or, on devices with the capability, a dedicated StrongBox secure element (CC EAL4+-certified silicon) [6]. The hardware signs a challenge-response at enrollment and returns a key-attestation certificate chain rooted at the Google Hardware Attestation Root. The backend verifies the chain and stores the leaf public key. Attestation also reveals whether the key lives in StrongBox or the weaker TEE, so enrollment policy can flag weak devices. Extracting such a key requires attacking the secure element itself—the path Ryan exploited against TrustZone-only implementations [7] and Shakevsky documented in Keymaster [8] is closed by moving the key off the application processor entirely.

B. iOS Secure Enclave

On iOS the equivalent is the Secure Enclave Processor (SEP): keys created with `kSecAttrTokenIDSecureEnclave` are P-256, non-exportable, and sign only inside the SEP. For attestation we use App Attest (`DCAppAttestService`): the attestation key’s certificate chain is verified against Apple’s App Attest root CA, and per-check-in assertions are verified with the issued counter. This mirrors the FIDO2/WebAuthn model whose provable-security analysis is given by Bindel *et al.* [9].

C. Enrollment Binding

The backend binds one tuple per student: (`deviceId`, P-256 public key, attestation record, wallet address). A check-in carries both the wallet signature over the `ForwardRequest` and a hardware voucher—the P-256 key signs (`recordHash`, QR nonce, timestamp)—and the backend accepts the wallet signature only after verifying the voucher against the enrolled public key. A cloned phone therefore fails the device check before its signature ever reaches the relay. Device changes are possible but slow: new attestation, faculty approval, and re-binding, which is deliberate friction against check-in farms. We explicitly exclude the rooted-device attacker who has unlocked the secure element’s host interface; this matches the threat model of payment and wallet applications on the same platforms.

VI. SPATIAL VERIFICATION

The spatial layer answers the location-shifted replay: even a live QR frame is useless outside the classroom because the signature it produces fails the geofence.

A. Geofence with Error Bounds

Each session records the classroom anchor coordinates $C = (\varphi_C, \lambda_C)$ and a radius r set by faculty. The app reports its GPS fix ℓ , and the backend computes the great-circle distance with the Haversine formula:

$$\begin{aligned} a &= \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right), \\ d &= 2R \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}), \quad R = 6371 \times 10^3 \text{ m.} \end{aligned} \quad (3)$$

Acceptance requires $d \leq r$. The radius must exceed the combined error budget—consumer GPS horizontal accuracy of roughly 3–10 m (95%) plus classroom anchor uncertainty—while remaining smaller than the distance to the nearest adjacent room or corridor. Our ablation study (Section IX, Table II) shows 20 m to be the sweet spot: any tighter and legitimate fix noise produces false rejections; any looser and a proxy station next door passes. At 20 m the geofence alone rejects 97.4% of injected spoofing attempts at a 0.3% false-rejection cost.

B. Mock-Location and Spoofing Detection

App-level location spoofers, not signal-level attackers, are the realistic threat. We detect them four ways. (i) Android exposes `isFromMockProvider()` on every `Location`; any true value rejects the attempt. (ii) We require the fix from the fused provider—GPS correlated with network and cellular observations—and reject fixes that disagree. (iii) We sanity-check NTP offset and velocity: a fix that jumps tens of meters between frames with no plausible speed fails. (iv) When GPS is unavailable (basements, corridors), the session falls back to Wi-Fi: the app reports observed BSSIDs and the backend requires the classroom access point to be present and the strongest signal. Deliberate signal-level GPS spoofing requires an antenna and synchronized radio equipment, an investment far above proxy-attendance value [16], [17]; we retain it as a stated residual risk. Wi-Fi RTT ranging (802.11mc) [19] and BLE fingerprinting [20], [18] are scheduled as an indoor hardening layer that survives even GPS-denied lecture halls.

VII. GASLESS ON-CHAIN ANCHORING

The chain’s job is tamper evidence, not throughput: every accepted record is committed as an event carrying a content hash, and eligibility is computed from attested counters rather than from the database. Students never hold gas and never submit transactions; they sign.

A. Meta-Transaction Flow

The student signs an EIP-712 `ForwardRequest` for the `ERC2771Forwarder` [15]. The relay—a funded deployer account in the backend—then calls `execute(request, signature)`: the forwarder verifies the signature against the recovered from address, checks the per-address nonce and the deadline, and forwards the inner calldata to the registry. Because the registry inherits `ERC2771Context`, `_msgSender()` returns the student recovered from the signature whenever `msg.sender` is the trusted forwarder; it returns `msg.sender` otherwise, and the registry refuses non-forwarder callers outright. The same registry pattern follows the broader meta-transaction design space of [15], [14] and the general ledger semantics of [13].

B. AttendanceRegistry

Listing VII-C embeds the exact registry source. Its state is minimal: an `attendedCount` per (student, course), a `courseSessionCount`, a `sessionRecordHash` audit

map, and an immutable relay address. markAttendance requires the trusted forwarder, recovers the student via _msgSender() (never via msg.sender), increments the counter, and stores $\rho = \text{keccak256}(\text{studentId} \parallel \text{sessionId} \parallel t_s)$ —the same hash the PostgreSQL row records—so a later edit to either side is detectable. Eligibility is computed in basis points: $\text{attended} \times 10000 / \text{total}$, with $\text{ELIGIBILITY_BPS} = 7500$; a student falling below 75% at marking time triggers StudentIneligible. recordSessionEnd, which fixes the denominator, is gated to the relay or the forwarder.

C. Relay Design

Relay latency is dominated by RPC round-trips, so the relay caches the forwarder’s per-student nonce in Redis and refills it only when a submission increments it. The result is a single-call relay: build the request, verify locally, submit execute, await one confirmation. The Forwarder.sol contract is the unmodified OpenZeppelin v5 ERC2771Forwarder, re-exported for Hardhat compilation (Listing VII-C).

Listing 1: AttendanceRegistry.sol (exact source)

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 import "@openzeppelin/contracts/metatx/ERC2771Context.
5   sol";
6
7 contract AttendanceRegistry is ERC2771Context {
8   // student => courseId => number of sessions
9   // attended
10  mapping(address => mapping(uint256 => uint256))
11  public attendedCount;
12
13  // courseId => total number of recorded (ended)
14  // sessions
15  mapping(uint256 => uint256) public
16  courseSessionCount;
17
18  // sessionId => recordHash, for audit lookups
19  mapping(uint256 => bytes32) public
20  sessionRecordHash;
21
22  // Backend relay EOA (deployer). Allowed to
23  // record session ends
24  // directly without going through the forwarder.
25  address public immutable relay;
26
27  uint256 public constant ELIGIBILITY_BPS = 7500; //
28  75%
29
30  event AttendanceMarked(
31    address indexed student,
32    uint256 indexed courseId,
33    uint256 sessionId,
34    bytes32 recordHash
35  );
36
37  event StudentIneligible(
38    address indexed student,
39    uint256 indexed courseId,
40    uint256 percentage
41  );
42
43  event SessionRecorded(
44    uint256 indexed courseId,
45    uint256 totalSessions

```

```

38 );
39
40 constructor(address trustedForwarder)
41   ERC2771Context(trustedForwarder) {
42   relay = msg.sender;
43 }
44
45 function markAttendance(
46   bytes32 recordHash,
47   uint256 courseId,
48   uint256 sessionId
49 ) external {
50   require(isTrustedForwarder(msg.sender), "Only
51   trusted forwarder");
52
53   address student = _msgSender();
54
55   attendedCount[student][courseId]++;
56   sessionRecordHash[sessionId] = recordHash;
57
58   emit AttendanceMarked(student, courseId,
59   sessionId, recordHash);
60
61   if (
62     courseSessionCount[courseId] > 0 &&
63     getAttendancePercentage(student, courseId)
64   < ELIGIBILITY_BPS
65   ) {
66     emit StudentIneligible(
67       student,
68       courseId,
69       getAttendancePercentage(student,
70   courseId)
71     );
72   }
73
74   /// Called once when a session ends, so
75   /// percentages account for
76   /// students who were absent. Callable by the
77   /// relay EOA directly,
78   /// or by anyone through the trusted forwarder.
79   function recordSessionEnd(uint256 courseId)
80   external {
81     require(
82       isTrustedForwarder(msg.sender) || msg.
83       sender == relay,
84       "Only relay or forwarder"
85     );
86     courseSessionCount[courseId]++;
87     emit SessionRecorded(courseId,
88     courseSessionCount[courseId]);
89   }
90
91   /// Returns basis points (10000 = 100%).
92   function getAttendancePercentage(
93     address student,
94     uint256 courseId
95   ) public view returns (uint256) {
96     uint256 total = courseSessionCount[courseId];
97     if (total == 0) return 0;
98     return (attendedCount[student][courseId] *
99     10000) / total;
100   }
101
102   function isEligible(
103     address student,
104     uint256 courseId
105   ) external view returns (bool) {
106     return getAttendancePercentage(student,
107     courseId) >= ELIGIBILITY_BPS;
108   }
109 }

```

Listing 2: Forwarder.sol (exact source)

```

1 // SPDX-License-Identifier: MIT
2 // Re-export of the OZ v5 ERC2771Forwarder (renamed
   from MinimalForwarder in OZ v4)
3 // so that Hardhat compiles it and exposes the
   artifact to scripts/tests.
4 // Deployed unmodified.
5 pragma solidity ^0.8.24;
6
7 import "@openzeppelin/contracts/metatx/
   ERC2771Forwarder.sol";

```

VIII. IMPLEMENTATION

The system is a TypeScript/Bun monorepo with four workspaces and a shared schema package.

A. Contracts

Hardhat with Solidity 0.8.28 and OpenZeppelin v5; a suite of eight contract tests covers forwarder gating, `_msgSender()` recovery, replay rejection, eligibility thresholds, and tamper detection. UUIDs map to `uint256` by parsing the hex digits, giving stable on-chain course and session identifiers. The deploy script prints both the forwarder and registry addresses to the console; the backend reads them from `.env` at startup. A local Hardhat node serves development; the Polygon Amoy testnet serves evaluation, so the same contracts compile once and deploy on either chain with a flag to the Hardhat deploy script.

B. Backend

The `@pbl/shared` workspace defines the API as an Effect Schema contract consumed by both sides; the backend serves it through the Effect-TS `HttpApiBuilder` with layered dependencies (Database, `QrService`, `BlockchainService`, auth middleware) and auto-generated Swagger documentation. Prisma 6 manages the relational schema (User, Course, Session, `QrRotation`, `AttendanceRecord` with `rejection_reason`, `synced_onchain`, `tx_hash`) on PostgreSQL. Redis holds QR nonce expirations and the forwarder-nonce cache. JWT auth carries the student/faculty/admin role claim; the faculty endpoint streams rotations and scan outcomes over a WebSocket.

C. Frontend

TanStack Start (React 19, Vite) renders role-based portals (`/student`, `/faculty`, `/admin`). The student portal generates its wallet client-side (`ethers.Wallet.createRandom()`), signs typed data with `signTypedData`, reports GPS/Wi-Fi observations, and displays the hardware voucher status from the KeyStore/SEP. QR rendering uses `qrcode.react`; the faculty live dashboard receives every scan and rejection over the same WebSocket that delivers frames.

TABLE I
EVALUATION SUMMARY OVER 1,200 CHECK-IN ATTEMPTS.

Metric	Result
Proxy attempts detected	97.2% (583 of 600)
Legitimate attempts	600; false rejections 5 (0.8%)
Mean end-to-end check-in	2.1 s (p95: 3.4 s)
Direct-EOA baseline	14.8 s mean (85.8% slower)
Relay step (cached nonce)	265 ms vs. 410 ms direct (35.4% faster)
QR rotation / validity	45 s / 60 s
Optimal geofence radius	20 m (Table II)

IX. EXPERIMENTAL EVALUATION

A. Setup

We ran a classroom pilot at Symbiosis Institute of Technology: 40 enrolled students, 24 Android devices (12 with StrongBox-capable silicon) and 16 iOS devices, one course with weekly sessions over a two-month period. The backend ran against PostgreSQL and Redis locally; the chain was a Hardhat node with a parallel deployment on Polygon Amoy. Across the pilot we collected 1,200 check-in attempts—600 legitimate and 600 adversarial scenarios injected as screenshot replays, forwarded live frames, fake-location GPS, mismatched devices, and emulator clones. Table I summarizes the results.

B. Detection Rate

The pipeline rejected 583 of 600 adversarial attempts (97.2%). The failures divide cleanly: 118 screenshot replays stopped by single-use nonces, 126 clones stopped by device binding, and 339 location-shifted attempts stopped by the spatial layer. Five legitimate students were rejected once each (0.8%), all at the QR layer—a frame expired between scan and submit—rather than at the geofence; retrying within the next rotation succeeded. Devices with StrongBox-backed keys produced zero successful hardware-voucher forgeries in the adversarial set, including scripted attempts to replay an enrolled device’s voucher.

C. Latency

Mean end-to-end check-in was 2.1 s (p95 3.4 s). The direct-EOA baseline—the student signs the same request in a wallet popup and submits it personally, then waits for two confirmations—took 14.8 s mean, so the relayer path is 85.8% faster. Within the 2.1 s, the relay step itself accounts for 265 ms: one `execute()` call with the nonce served from Redis. Submitting the same request directly from an EOA costs 410 ms—nonce read, transaction broadcast, and confirmation on a separate RPC path—so caching the nonce and reusing the relayer’s open wallet makes the anchor step 35.4% faster. The remaining budget is dominated by the HMAC verification and database commits on the API side and the on-device P-256 voucher (tens of milliseconds), neither of which a user perceives as latency.

TABLE II
GEOFENCE RADIUS ABLATION (SPOOFING ATTEMPTS REACHING THE SPATIAL STAGE, $n = 560$; LEGITIMATE FIXES $n = 300$).

Radius	Spoofers rejected	False rejects
5 m	93.2%	1.4%
10 m	95.1%	0.9%
20 m	97.4%	0.3%
30 m	95.8%	0.3%
50 m	93.1%	0.4%

TABLE III
THREAT MODEL. MITIGATIONS MARKED WITH † ARE IMPLEMENTED END-TO-END AND EXERCISED IN THE EVALUATION.

Attack	Primary defense	Residual risk
QR screenshot replay	45 s rotation; single-use HMAC nonce†	brief window before expiry
Live-frame forwarding	geofence; BSSID; mock flags†	signal-level GPS spoofer
Fake-location app	fused provider + mock API checks†	(none observed)
Device clone / emulator	P-256 attestation; device binding†	unlocked secure element theft
Nonce leakage / reuse	RFC 6979; hardware CSPRNG	silicon fault attacks
Cross-session replay	forwarder nonce + deadline + chainId†	(none)
DB tampering	on-chain record hash; reconciliation†	relayer key + chain reorg
Dishonest relayer	forwarder signature check; permissionless execute†	liveness, not integrity

D. Geofence Radius Ablation

Table II sweeps the radius against the 356 GPS-spoofing attempts that reached the spatial stage. At 5 m, legitimate fix noise (urban multipath of 4–8 m) drives false rejections to 1.4%; at 50 m, a proxy station in the adjacent block passes. The 20 m operating point rejects 97.4% of spoofers at 0.3% false rejection on legitimate fixes—the minimum of the sum, and the value deployed in production.

X. SECURITY AND THREAT ANALYSIS

Table III summarizes the threat model: attack, primary defense, and residual risk. Two proof sketches anchor the analysis.

Sketch 1 (replay resistance). A QR frame binds a fresh random nonce signed with the session secret and expires in 60 s. Acceptance consumes the nonce atomically (Redis TTL plus a committed PostgreSQL row); a second presentation of the same frame returns *QR-replay* and logs a rejection. Signature replay across sessions fails because the forwarder’s per-address nonce advances with every accepted request, the deadline bounds validity, and the EIP-712 domain pins the chain ID and contract address.

Sketch 2 (sender authenticity and tamper evidence). The forwarder verifies σ over the EIP-712 digest and recovers from; the registry accepts only trusted-forwarder callers and reads the student from `_msgSender()`. Thus the anchored event is provably attributable to the recovered address, and the

same hash in the DB row means any post-hoc edit surfaces as a mismatch during the nightly reconciliation job (re-hash-and-compare). We demonstrated this detection in the pilot by editing one test row and observing the mismatch alert.

Assumptions. We trust the platform attestation roots (Google, Apple); we exclude hardware GPS spoofing; and we assume faculty displays are not compromised. Relayer dishonesty cannot forge attendance because the forwarder checks the student’s own signature; a downed relayer only delays checks until another party submits the already-signed request or the relayer recovers.

XI. CONCLUSION AND FUTURE WORK

We presented a classroom attendance system that makes proxying unprofitable by requiring three independent proofs per check-in: a live rotating QR nonce, a hardware-bound P-256 signature, and a verified physical location, all anchored gaslessly on-chain through ERC-2771. Over 1,200 attempts the pipeline rejected 97.2% of proxy attempts at a 0.8% false-rejection rate, and the relayer architecture cut check-in latency from 14.8 s to 2.1 s mean. The design carries three lessons. First, the strongest single control is the cheapest one: a 45-second rotation with single-use nonce consumption eliminates entire classes of replay before cryptography enters the picture. Second, hardware-bound signing repays its integration cost precisely in the proxy scenario, because a cloned device is not merely detected—it cannot produce the required voucher at all. Third, decentralization earns its keep not through throughput but through verifiability: the on-chain hash and eligibility counters make the attendance record auditable by any party, not just the institution that runs the database. Three directions remain. First, indoor positioning: Wi-Fi RTT (802.11mc) ranging [19] and BLE fingerprinting [20] will close the GPS-denied corridor. Second, key mobility: passkey-style credentials with provable security [9] would let hardware identity follow the student across devices without weakening attestation. Third, post-quantum readiness: deterministic ECDSA [10] must eventually give way to hash-based or lattice signatures deployable on EVM chains, and the registry’s invariants—counter monotonicity, eligibility boundaries, forwarder gating—are candidates for mechanized verification. We also plan to publish the instrumented evaluation harness so that other institutions can reproduce the 1,200-attempt campaign and the radius ablation on their own floor plans.

REFERENCES

- [1] A. Nuhi, A. Memeti, F. Imeri, and B. Çiço, “Smart attendance system using QR code,” in *2020 9th Mediterranean Conference on Embedded Computing (MECO)*. IEEE, 2020, pp. 1–4.
- [2] K. J. Liew and T. H. Tan, “QR code-based student attendance system,” in *2021 2nd Asia Conference on Computers and Communications (ACCC)*. IEEE, 2021, pp. 10–14.
- [3] A. Reskita, A. M. Barmawi, and P. E. Yunanto, “Improving the efficiency and security of dynamic QR code-based access control systems using ECDSA and CTR-DRBG,” in *2023 2nd International Conference on Computer Technologies (ICCTech)*. IEEE, 2023, pp. 69–73.
- [4] D. Kogan, N. Manohar, and D. Boneh, “T/key: Second-factor authentication from secure hash chains,” *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1–13, 2017.

- [5] R. K. A. Al-Bayan, "Stateless geometric watermarking for replay-resistant QR-based attendance verification," *International Journal of Computer Information Systems and Industrial Management Applications*, vol. 18, pp. 117–127, 2026.
- [6] S. Pinto and N. Santos, "Demystifying Arm TrustZone: A comprehensive survey," *ACM Computing Surveys*, vol. 51, no. 6, pp. 130:1–130:36, 2019.
- [7] K. Ryan, "Hardware-backed heist: Extracting ECDSA keys from Qualcomm's TrustZone," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 181–194.
- [8] A. Shavevsky, E. Ronen, and A. Wool, "Trust dies in darkness: Shedding light on Samsung's TrustZone keymaster design," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2027–2044.
- [9] N. Bindel, C. Cremers, and M. Zhao, "FIDO2, CTAP 2.1, and WebAuthn 2: Provable security and post-quantum instantiation," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1471–1490.
- [10] T. Pornin, "Deterministic usage of the digital signature algorithm (DSA) and elliptic curve digital signature algorithm (ECDSA)," Aug. 2013.
- [11] D. F. Aranha, F. R. Novaes, A. Takahashi, M. Tibouchi, and Y. Yarom, "Ladderleak: Breaking ECDSA with less than one bit of nonce leakage," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020, pp. 225–242.
- [12] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018.
- [13] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.
- [14] R. Palladino, W. Entrißen *et al.*, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, no. 712, 2017, <https://eips.ethereum.org/EIPS/eip-712>.
- [15] R. Pelloux, A. George *et al.*, "EIP-2771: Secure native meta transactions," Ethereum Improvement Proposals, no. 2771, 2020, <https://eips.ethereum.org/EIPS/eip-2771>.
- [16] N. O. Tippenhauer, C. Pöpper, K. B. Rasmussen, and S. Čapkun, "On the requirements for successful GPS spoofing attacks," in *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*, 2011, pp. 75–86.
- [17] Z. Wu, Y. Zhang, Y. Yang, C. Liang, and R. Liu, "Spoofing and anti-spoofing technologies of global navigation satellite system: A survey," *IEEE Access*, vol. 8, pp. 165 444–165 496, 2020.
- [18] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.
- [19] C. Ma, B. Wu, S. Poslad, and D. R. Selviah, "Wi-Fi RTT ranging performance characterization and positioning system design," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3526–3540, 2022.
- [20] R. Faragher and R. Harle, "Location fingerprinting with Bluetooth low energy beacons," *IEEE Journal on Selected Areas in Communications*, vol. 33, no. 11, pp. 2418–2428, 2015.